

CODE REVIEW & CI/CD

Claude Code GitHub Actions

 Copy page

Run Claude Code in GitHub Actions workflows to respond to @claude mentions, automate tasks, and turn issues into pull requests

Claude Code GitHub Actions is a GitHub Action that runs Claude Code inside your repository's workflows. Mention `@claude` in a pull request or issue comment to have Claude analyze code, implement changes, and push commits. You can also give the Claude Code GitHub Action a prompt to run automatically on any GitHub event. Use it to turn issues into pull requests, fix bugs from a comment, or automate recurring tasks.

Several products share the Claude Code name. This page covers the `claude-code-action` workflow integration, which you configure with workflow files in your repository. For the related products, see:

- **Code Review**: automatic review on every pull request, without writing a workflow
- **Claude Code on the web**: Claude Code sessions from your browser or phone
- **Claude Agent SDK**: custom automation outside GitHub Actions. The Claude Code GitHub Action is built on the SDK
- **GitHub Enterprise Server**: Claude Code with self-hosted GitHub

Setup

You can set up the Claude Code GitHub Action in one of two ways:

- **Quick setup**: run `/install-github-app` from Claude Code. Claude Code installs the GitHub App, adds your authentication secret, and prepares the workflow pull request for you

For either path, you need admin access to the repository.

Quick setup

Before you start, install the **GitHub CLI** and authenticate it with `gh auth login`. Claude Code checks for it and warns you if it's missing.

Open `claude` in the repository you want to connect, run `/install-github-app`, and follow the prompts. Claude Code installs the Claude GitHub App, then sets up an authentication secret for the workflows:

- If Claude Code already has an API key, it reuses that key, and offers to keep the repository's existing `ANTHROPIC_API_KEY` secret if one is already set
- Otherwise, choose between creating a long-lived token with your Claude subscription and pasting in an API key

Claude Code saves the credential as a repository secret, named `ANTHROPIC_API_KEY` for an API key or `CLAUDE_CODE_OAUTH_TOKEN` for a subscription token.

Claude Code then pushes a branch with the workflow files you select, already set to use that secret, and opens GitHub in your browser with a pull request ready to create. Create and merge that pull request, and `@claude` works in the repository.

If you select the review workflow, Claude posts each review on the pull request itself, as an inline comment on each issue it finds or as one summary comment when it finds none. Claude skips some pull requests, such as drafts. The [review workflow example](#) uses the same skill and lists them. Before v2.1.229, Claude wrote its review only to the workflow run log.

To update a review workflow that an earlier version generated, do one of the following:

- Add the `--comment` argument and the `claude_args` line from the [review workflow example](#) to the checked-in file yourself, which keeps any other edits you made to it.

After installing the GitHub App, Claude Code asks whether to continue with GitHub Actions setup. Choose **Skip for now** to stop with only the GitHub App installed. Run `/install-github-app` again later to finish the workflow and secret steps. Before v2.1.187, Claude Code proceeded straight to workflow selection.



- When you install the GitHub App, you grant it several permissions. See [GitHub App permissions](#) for the full set
- Quick setup works with the Claude API and Claude subscriptions. If you use Amazon Bedrock, Google Cloud's Agent Platform, or Microsoft Foundry, see [Use Claude Code GitHub Actions with cloud providers](#)

Manual setup

To configure the Claude Code GitHub Action without running `/install-github-app`, install the app, add a secret, and copy a workflow file yourself:

- **Contents:** read and write, so Claude can modify repository files
- **Issues:** read and write, so Claude can respond to issues
- **Pull requests:** read and write, so Claude can create PRs and push changes

During installation, you also grant permissions that other Claude features use. See [GitHub App permissions](#) for the full set.

2 Add an authentication secret

Add one of the following secrets to your repository, depending on how you authenticate. See GitHub's guide to [using secrets in GitHub Actions](#).

- `ANTHROPIC_API_KEY` : a Claude API key from the [Claude Console](#)
- `CLAUDE_CODE_OAUTH_TOKEN` : an OAuth token that authenticates with your Claude subscription, available on Pro, Max, Team, and Enterprise plans. Generate one by running `claude setup-token` locally. See [Generate a long-lived token](#)

In workflow files, pass the secret to the matching input: `anthropic_api_key` for an API key, or `claude_code_oauth_token` for an OAuth token.

3 Copy the workflow file

Copy [examples/claude.yml](#) into your repository's `.github/workflows/` directory. The file is a working workflow, not just an example. As committed, Claude responds whenever someone mentions `@claude` in an issue or pull request, authenticating with the `ANTHROPIC_API_KEY` secret. If you added `CLAUDE_CODE_OAUTH_TOKEN` instead, change the workflow's `anthropic_api_key` line to `claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }`.

Set up for an organization

With quick setup or manual setup, you configure one repository at a time. To roll the Claude Code GitHub Action out across an organization:

- Install the **Claude GitHub App** once at the organization level, choosing all repositories or a selected list
- Store the authentication secret as an organization-level Actions secret so each repository doesn't need its own copy
- Add the workflow file to each repository that should run the Claude Code GitHub Action, or define the job once as a **reusable workflow** that each repository calls

For a secret shared across repositories, authenticate with an API key from the **Claude Console** rather than an OAuth token, since an OAuth token is tied to the subscription of the person who ran `claude setup-token`.

To avoid storing a long-lived secret entirely, authenticate through workload identity federation, where the Claude Code GitHub Action exchanges the workflow's GitHub OpenID Connect (OIDC) token for Claude API access through a Claude Console service account. Set these inputs:

- `anthropic_federation_rule_id` : the federation rule ID, `fdr1_...`
- `anthropic_organization_id` : your Anthropic organization ID
- `anthropic_service_account_id` : the service account ID, `svac_...` . Optional, since the federation rule you create in the Console already targets a service account
- `anthropic_workspace_id` : the workspace ID, `wrkspc_...` . Optional when the federation rule targets a single workspace

Grant the workflow the `id-token: write` permission, which the Claude Code GitHub Action needs for the federation exchange even when you pass your own `github_token` . See the **Claude Code GitHub Action's setup guide** for the Console-side configuration.

To remove the Claude Code GitHub Action, undo each piece of the setup that applies to your installation:

- **Workflow files:** delete the workflows that use `anthropics/claude-code-action` from `.github/workflows/`. If you used quick setup, look for `claude.yml` and, if you selected the review workflow, `claude-code-review.yml`. With the workflows deleted, the Claude Code GitHub Action no longer runs
- **Secrets:** delete the `ANTHROPIC_API_KEY` or `CLAUDE_CODE_OAUTH_TOKEN` secret from the repository, and from organization-level Actions secrets if you shared it across repositories. If you delete a secret, the credential it held stays valid. To retire an API key entirely, also delete the key in the Claude Console
- **GitHub App:** uninstall the Claude GitHub App in your repository or organization settings under GitHub Apps, but only if you don't use it for another Claude feature, such as Code Review or web auto-fix

If you configured a cloud provider, also delete the provider secrets, such as `AWS_ROLE_TO_ASSUME`, the `GCP_*` secrets, or the `AZURE_*` secrets, and uninstall the custom GitHub App along with its `APP_ID` and `APP_PRIVATE_KEY` secrets.

GitHub App permissions

The Claude GitHub App is shared by every Claude feature that integrates with GitHub, including the Claude Code GitHub Action, Code Review, and auto-fix for pull requests on Claude Code on the web. A GitHub App has a single permission set covering all of its features, so the set includes some permissions that the Claude Code GitHub Action doesn't use.

When you install the app, you grant the following permissions:

Permission	Access
Actions	Read and write

Discussions	Read and write
Issues	Read and write
Members	Read
Metadata	Read
Pull requests	Read and write
Repository hooks	Read and write
Statuses	Read
Workflows	Read and write

The permission set can also change ahead of the features that use it. When the app requests a permission it didn't have before, GitHub prompts the account owner to approve it, an organization owner for an organization install, and the installation keeps its old permissions until they do. For example, when Actions access changes from read to write, the app can re-run workflows rather than only view runs and logs, so GitHub asks the owner to approve the change.

When you install the app, you accept its full permission set. GitHub doesn't let you accept a subset. If your organization requires only the permissions the Claude Code GitHub Action uses, create a custom GitHub App with Contents, Issues, and Pull requests instead, following the [Claude Code GitHub Action's setup guide](#). A custom app covers only the Claude Code GitHub Action. Code Review and web auto-fix still require the official app.

For details on how the Claude Code GitHub Action limits what Claude can do with these permissions, see the [security documentation](#).

Interactive and automation modes

The Claude Code GitHub Action detects how to run from your workflow configuration:

- **Automation mode:** when the workflow provides a `prompt` input, Claude runs without waiting for a mention, subject only to the [checks on who can trigger runs](#). By default, results appear in the workflow run log rather than a comment. Claude can post to the issue or pull request when the prompt directs it to and it has a tool that can post, as in the [code-review example](#).

Who can trigger runs

In both modes, the Claude Code GitHub Action runs two checks on the triggering actor before Claude starts, and the run fails when either check rejects it:

- **Write access:** on issue and pull request events, the triggering user must have write access to the repository. To allow specific users without write access, set `allowed_non_write_users` and pass your own `github_token` input. Events that no user authors, such as a `schedule` trigger, skip this check.
- **Human actor:** on every event, the Claude Code GitHub Action rejects a bot actor unless you list it in `allowed_bots`, which keeps bots from triggering Claude in a loop. This check also applies to scheduled runs, which GitHub attributes to a repository user, usually the one who last changed the workflow's `cron` schedule. If that user is a bot, list it in `allowed_bots`.

Example use cases

The [examples directory](#) contains ready-to-use workflows for different scenarios.

The examples on this page show API key authentication. If you authenticate with a Claude subscription, replace the `anthropic_api_key` line in any example with `claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }`.

```
name: Claude Code
on:
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]
jobs:
  claude:
    if: contains(github.event.comment.body, '@claude')
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
      issues: write
      id-token: write
      actions: read
    steps:
      - uses: actions/checkout@v6
        with:
          fetch-depth: 1
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
```

The parts of this workflow that aren't boilerplate:

- `id-token: write` : required for the Claude Code GitHub Action's default GitHub App authentication
- `actions: read` : lets Claude read CI results on PRs
- `actions/checkout` : gives Claude a local copy of the repository to work in
- `if` : keeps runners from starting on comments that don't mention `@claude` . The Claude Code GitHub Action also checks the trigger phrase itself before responding

```
@claude how should I implement user authentication for this endpoint?  
@claude fix the TypeError in the user dashboard component
```

Claude replies in a comment on the same issue or PR and updates it as it works.

Run a skill

The `prompt` input accepts a skill invocation as well as plain text:

- For a skill in your repository's `.claude/skills/` directory, run `actions/checkout` before the `anthropics/claude-code-action` step so the skill files are available on the runner, then pass `/skill-name` as the `prompt`.
- For a skill packaged in a plugin, install the plugin with the `plugin_marketplaces` and `plugins` inputs, then pass the namespaced `/plugin-name:skill-name` as the `prompt`. The `plugins` input takes `plugin-name@marketplace-name`, where the marketplace name comes from the marketplace's own manifest rather than its repository URL.

The following workflow installs the `code-review` plugin and runs its skill when a pull request is opened, updated, reopened, or marked ready for review. It runs the same plugin as the review workflow from quick setup. Use a workflow like this when you want to control the prompt, model, and triggers yourself. For automatic reviews without maintaining a workflow file, see [Code Review](#). On public repositories, GitHub withholds secrets from runs triggered by fork pull requests, so the review runs only on pull requests from branches in the same repository.

```
jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: read
      issues: read
      id-token: write
    steps:
      - uses: actions/checkout@v6
        with:
          fetch-depth: 1
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          plugin_marketplaces: "https://github.com/anthropics/claude-code.git"
          plugins: "code-review@claude-code-plugins"
          prompt: "/code-review:code-review --comment ${ github.repository_owner }"
          claude_args: '--allowedTools "mcp__github_inline_comment__create_i'
```

Two lines in this workflow control where the review goes:

- `--comment` : Claude posts its review on the pull request, as an inline comment on each issue it finds or as one summary comment when it finds none. Without it, Claude posts nothing, and you read the findings in the workflow run log.
- `claude_args` : keep this line even though the skill's own `allowed-tools` frontmatter names the same tool, because the Claude Code GitHub Action starts the MCP server that posts inline comments only when `--allowedTools` in `claude_args` names it.

Claude skips draft and closed pull requests, pull requests it judges not to need a review, such as automated or trivial ones, and pull requests that already have a comment from Claude.

`claude_args` or a `permissions.allow` **rule** in the `settings` input. If you invoke a skill instead, Claude can use the tools its `allowed-tools` **frontmatter** grants. GitHub runs scheduled workflows only from the default branch and, in public repositories, disables the schedule after 60 days without repository activity.

This workflow generates a report in the workflow run log at 09:00 UTC each day. Its `claude_args` line **passes CLI arguments** that select the model and allow two GitHub MCP tools. Claude reads commits and issues through the GitHub API with those tools, so you can omit the checkout step:

```
name: Daily Report
on:
  schedule:
    - cron: "0 9 * * *"
jobs:
  report:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      issues: read
      id-token: write
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: "Generate a summary of yesterday's commits and open issues"
          claude_args: |
            --model claude-opus-4-8
            --allowedTools "mcp__github__list_commits,mcp__github__list_issues"
```

Create a `.CLAUDE.md` file in your repository root to define code style guidelines, review criteria, project-specific rules, and preferred patterns. Claude follows these guidelines when creating PRs and responding to requests. See the [memory documentation](#) for details.

Protect your credentials

⚠️ Never commit API keys or OAuth tokens directly to your repository. Always store them as GitHub Secrets and reference them in workflows, for example `anthropic_api_key: ${secrets.ANTHROPIC_API_KEY}`.

Grant the workflow only the permissions it needs, and review Claude's changes before merging.

For comprehensive security guidance including permissions and authentication, see the [Claude Code Action security documentation](#).

Manage costs

Each run consumes two kinds of resources:

- **GitHub Actions minutes:** the Claude Code GitHub Action runs on GitHub-hosted runners, which consume your GitHub Actions minutes. See [GitHub's billing documentation](#) for pricing and minute limits.
- **API tokens:** each interaction consumes tokens based on the length of prompts and responses, task complexity, and codebase size. See [Claude's pricing page](#) for current token rates. If you authenticate with an OAuth token, runs use your Claude subscription instead of API billing.

You can lower both kinds of cost by giving Claude clearer context and by capping how much work each run can do:

- Write specific `@claude` requests so Claude needs fewer turns to finish

- Set workflow-level timeouts to avoid runaway jobs
- Use GitHub's concurrency controls to limit parallel runs

For usage tracking across your organization, see the [analytics dashboard](#) and [monitoring](#). For how usage is measured and billed, see [costs](#).

Use a cloud provider

By default, the Claude Code GitHub Action calls the Claude API directly with your API key or OAuth token. To route inference through your own cloud account instead, set the input for your provider and follow [Use Claude Code GitHub Actions with cloud providers](#):

- **Amazon Bedrock:** `use_bedrock: "true"`
- **Google Cloud's Agent Platform:** `use_vertex: "true"`
- **Microsoft Foundry:** `use_foundry: "true"`

With all three providers, you authenticate through OIDC identity federation instead of a Claude API key, so you store no static cloud credentials in your repository.

Troubleshooting

Claude not responding to @claude commands

- Verify the GitHub App is installed on the repository
- Check that workflows are enabled for the repository
- Ensure your API key or OAuth token is set in repository secrets
- Confirm the comment contains `@claude` as a complete word, not `/claude` or `@claude-bot`

- GitHub doesn't trigger workflows on commits made with the default `GITHUB_TOKEN`. If you pass `github_token: ${ secrets.GITHUB_TOKEN }` to the Claude Code GitHub Action, remove it so it authenticates as the Claude GitHub App, or pass a custom app token instead
- Check that your CI workflow's triggers include the events Claude's pushes produce, such as `push` or `pull_request`

Authentication errors

- Confirm the API key or OAuth token is valid by testing it locally with `claude` before debugging the workflow
- For Bedrock, Agent Platform, and Foundry, see the cloud provider page's [troubleshooting section](#)

For more solutions, see the Claude Code GitHub Action's [FAQ](#).

Advanced configuration

Action parameters

These are the most commonly used inputs. Each maps to a `with:` key in the `anthropics/claude-code-action` step.

Parameter	Description	Required
<code>prompt</code>	Instructions for Claude, as plain text or a skill invocation. When omitted, Claude responds to the trigger phrase instead	No
<code>claude_args</code>	CLI arguments passed to Claude Code	No

		Foundry
<code>claude_code_oauth_token</code>	OAuth token for authenticating with a Claude subscription, generated with <code>claude setup-token</code>	No
<code>github_token</code>	Token for GitHub operations. When omitted, the Claude Code GitHub Action authenticates as the Claude GitHub App	No
<code>plugin_marketplaces</code>	Newline-separated list of plugin marketplace Git URLs	No
<code>plugins</code>	Newline-separated list of plugin names to install before execution	No
<code>settings</code>	Claude Code settings, as a JSON string or a path to a settings JSON file	No
<code>trigger_phrase</code>	Trigger phrase Claude responds to. Default: <code>@claude</code>	No
<code>use_bedrock</code>	Use Amazon Bedrock instead of the Claude API	No
<code>use_vertex</code>	Use Google Cloud's Agent Platform instead of the Claude API	No
<code>use_foundry</code>	Use Microsoft Foundry instead of the Claude API	No

For the full input list, see the Claude Code GitHub Action's [configuration reference](#).

Pass CLI arguments

The `claude_args` parameter accepts any [Claude Code CLI argument](#):

```
claude_args: "--max-turns 5 --model claude-sonnet-5 --mcp-config /path/to/c
```

Claude Code GitHub Action uses the Claude Code default model

- `--mcp-config` : path to MCP configuration
- `--allowedTools` : comma-separated list of allowed tools. The `--allowed-tools` alias also works
- `--debug` : enable debug output

Upgrade from beta

If your workflows still reference `anthropics/claude-code-action@beta`, update them to v1:

1. Change `@beta` to `@v1` in the `uses` line
2. Remove the `mode` input, since the Claude Code GitHub Action now detects the mode automatically
3. Replace `direct_prompt` with `prompt`
4. Move CLI options such as `max_turns` and `model` into `claude_args`.
`custom_instructions` has no same-name flag and becomes `--append-system-prompt`

For the full input mapping and before-and-after examples, see the migration guide.

What's next

- Use Claude Code GitHub Actions with cloud providers: route inference through Amazon Bedrock, Google Cloud's Agent Platform, or Microsoft Foundry
- Configuration reference: the full list of action inputs
- Examples directory: ready-to-use workflows for more scenarios
- Code Review: automatic pull request review without maintaining a workflow file

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)